Homework #1: Manual Test-Case Generation

Due date: 2026-05-05 (23:59)

In the field of software engineering, generating test cases for a target program is a common yet important task. Through this assignment, we want each student to gain hands-on experience in test-case generation.

The goal of this homework is to **manually generate test-cases that maximize code coverage (e.g., the number of covered branches**), which is a widely used metric for evaluating the quality of the test-cases. Let's use the example in Figure 1 to understand how branch coverage is measured. Generally, the number of branches in a program is twice the number of if (or while) statements. Since there are two if statements in the example, the total number of branches is 4. We only need two test cases to cover all the branches in Figure 1. For example, the first test case, where x = 90 and y = 25, will cover the two true branches. The second test case, where x = 0 and y = 0, will cover the two false branches. Together, these two test cases successfully cover all four branches. In our homework, we try to generate test-cases that increase the number of covered branches for a real-world open-source C program.

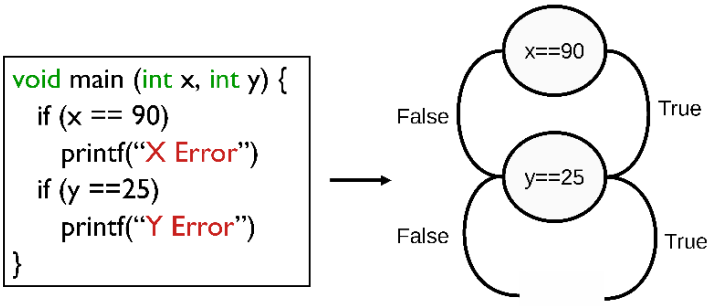

Figure 1 Examples for the number of covered branches

The target program for this homework is '**bison'**, a parser generator that takes a context-free grammar as input and generates a C program capable of parsing and processing structured input. Your goal is to execute the **bison** program with manually written test cases to maximize the number of branches covered during execution. The more branches your test cases cover, the higher your homework score will be. To measure branch coverage, we will use **gcov**, a widely used coverage analysis tool. It will automatically calculate how many branches of the program were executed with your test cases. To help you design more effective test cases for **bison**, we also provide a brief guide that explains how the program works and which features you can test to increase coverage.

You may submit up to 30 test cases in total (1 ≤ number of test cases ≤ 30). The number of branches covered by your test cases may vary, and your score will be based on how many branches are successfully covered. Your task is to write test cases for the **bison** program. Each test case should be a single command-line input (examples in example_testcases.txt), and all of your test cases must be written in studentid_testcases.txt. We will provide detailed instructions on downloading the homework files and running the **bison** program. Please follow those instructions carefully. If you have any questions, feel free to contact the TA via email or iCampus message.

Version & Environment

- Benchmark: bison-3.8
- Evaluation Environment: **Linux Ubuntu-22.04 LTS**
- You can get more information about bison at https://www.gnu.org/software/bison/manual/bison.html

Grading

- Test cases will be evaluated **three times**, and your final coverage score will be the **average** of those three runs.
- Coverage rates will be sorted in **descending order**, and scores will be assigned in groups of 10 students, with the top group receiving 10 points and each subsequent group receiving one point less.
- If your test cases **cause the program to crash**, you will receive an **additional 2 points** regardless of your ranking.
- Each test case must finish execution within **5 seconds**; otherwise, the command will be considered **timed out** and **ignored** during evaluation.
- Since the number of covered branches can vary across Ubuntu versions, the evaluation will be conducted **based on the environment described in this document**.
- Each test case must be written as a single-line command and must not exceed **300 characters**.
- There is no restriction on how the command is constructed (e.g., using echo, pipeline, or input files), but you may **only use files included in the original source code and generated by your test cases** (e.g., a file created in testcase1 can be reused in testcase2).
- The **bison** program must run once per test-case. (not allowed commands like ./**bison** .. && ./**bison**)
- Cases that modify environment variables using 'export' or similar methods are not permitted.

Submission

- just submit **studentid_testcases.txt** and upload it to i-campus (change **studentid** to your id).
- Due date: 2026-05-05, 23:59 (No late submission.)

** Caution: If Windows newline "\r" is in testcases, they return errors. Do not write your testcases file on Windows OS.

Instructions

1. Build a benchmark

First, download the *.zip* file from i-campus and extract the *.zip* file.

```
$ unzip 2026s_hw1.zip
```

After the given *.zip* file extraction,

```
$ cd 2026s_hw1
$ python3 script.py --build
```

** "--build" option builds bison.

If the build succeeds, the terminal will display the screen shown below.

```
  CC       lib/malloc/libbison_a-scratch_buffer_dupfree.o
  CC       lib/malloc/libbison_a-scratch_buffer_grow.o
  CC       lib/malloc/libbison_a-scratch_buffer_grow_preserve.o
  CC       lib/malloc/libbison_a-scratch_buffer_set_array_size.o
  CC       lib/unistr/libbison_a-u8-mbtoucr.o
  CC       lib/unistr/libbison_a-u8-uctomb.o
  CC       lib/unistr/libbison_a-u8-uctomb-aux.o
  CC       lib/uniwidth/libbison_a-width.o
  AR       lib/liby.a
  AR       lib/libbison.a
  CCLD     src/bison
  GEN      doc/bison.help
make[2]: Leaving directory '/home/user/2026s_hw1/bison-3.8'
make[1]: Leaving directory '/home/user/2026s_hw1/bison-3.8'
Build Complete
user@DESKTOP-EVUNGVF:~/2026s_hw1$
```

2. Make test-cases

Open studentid_testcases.txt and write testcases one per line. In that text file, there will be two examples, and you can choose to include in or exclude from your final set of 30 test cases.

3. Check the total coverage of your testcases

```
$ cd ~/2026s_hw1/
$ python3 script.py --testcase_file studentid_testcases.txt
```

** The way script.py works **

1. If you give ***studentid_testcases.txt*** as input, ***script.py*** will run all the commands in the ***studentid_testcases.txt*** file.

   e.g. ./bison --help

   e.g. ./bison --version

2. After a binary file is executed (by each testcase), *.gcda* files are automatically created for source codes in ***2026s_hw1/bison-3.8/src.***

   (cannot read *.gcda* file with "cat" or "vi" command)

3. Then it executes gcov and gcov creates *.gcov* file by reading the created *.gcda* files.
4. Finally, it reads the created *.gcov* files and calculates the number of covered branches.

```
Report bugs to <bug-bison@gnu.org>.
GNU Bison home page: <https://www.gnu.org/software/bison/>.
General help using GNU software: <https://www.gnu.org/gethelp/>.
Report translation bugs to <https://translationproject.org/team/>.
For complete documentation, run: info bison.
bison (GNU Bison) 3.8
Written by Robert Corbett and Richard Stallman.

Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
---------------Results-----------------------------------------
The number of covered branches: 208
The number of total branches: 14386
---------------------------------------------------------------
```

** Your goal is to increase the number of covered branches!! **

After checking the results, you just submit a text file.

---

4. How to check the coverage of your each testcase

```
$ cd 2026s_hw1/bison-3.8/src
$ ./bison --help
$ find . -name "*.gcda" -exec gcov -b {} \;
```

Through this, you can check the number of covered branches for each test-case.

- In this case, `./bison --help` is the testcase (command).
- That command (testcase) creates the *.gcda* file for each source codes in `2026s_hw1/bison-3.8/src` directory.
- Last command will check the number of covered branches by creating the *.gcov* files from *.gcda* files

```
Lines executed:0.00% of 182
File '/home/user/2026s_hw1/bison-3.8/src/main.c'
Lines executed:17.74% of 124
Branches executed:10.00% of 40
Taken at least once:5.00% of 40
Calls executed:13.04% of 92
Creating 'main.c.gcov'
```

** The number of covered branches is calculated as follows **

For each file,

Taken at least once: 5.00% (← covered branch ratio) of 40 (← the number of total branches)

*The number of covered branches = 0.05 * 40 = 2*

→ And we calculate the number of covered branches for all files and **add them together**.

5. How to check the covered branches in each source code by your testcases

```
$ cd 2026s_hw1/bison-3.8/src
$ ./bison --help
$ find . -name "*.gcda" -exec gcov -b {} \;
$ vi main.c.gcov
```

- Command creates *.gcda* file
- gcov creates *.gcov* file, which you can read by the “vi” or “cat” command

- *.gcov* file shows you which branch is covered
- You can search “branch” keyword in *.gcov* file

```
        1:   68:    if (cp)
branch  0 taken 0% (fallthrough)
branch  1 taken 100%
```

In line 74: if (cp) → (*in line 68 in main.c.gcov*, if statement’s branch 0, branch 1)

- branch 0 taken 0% → means this branch is not covered
- branch 1 taken 100% → means this branch is covered

```
    #####:  103:  if (trace_flag)
branch  0 never executed
branch  1 never executed
```

In line 127: if (trace_flag) → (*in line 103 in main.c.gcov*, if statement’s branch 0, branch 1)

- branch 0 never executed → means this branch is never executed
- branch 1 never executed → means this branch is never executed